

```

* in sys_call_table array for the given system call
handler.
* This is all, now system call handler will be called
*/

```

```

movq %r10, %rcx
call *sys_call_table(, %rax, 8)

```

Exiting from system call and resuming execution in user space

We have seen how the kernel executes the entry code for system call handler, saves user space program context onto the kernel stack, prepares stack frame for system call handle and, finally, executes the system call. Let's now find out how the kernel sends the return value of system call to the user program and resumes execution in the user mode.

Once the system call handler finishes the execution, control will again return to *arch/x86/entry/entry_64.S* right after where we have called the system call handler: *call *sys_call_table(, %rax, 8)*. Now let's examine the code given below to understand the steps for returning from a system call.

arch/x86/entry/entry_64.S

```

/* system call handler set the return value in rax register
* this rax value needs to be moved in kernel stack
* where user mode rax is saved
* After this all general purpose registers are restored
except rcx and r11
* rcx will be loaded with return address to the user mode
program i.e ip when
* user mode program triggers syscall instruction.
* This ip is saved in kernel stack
* r11 is loaded with flag registers of user mode programs.
*/

```

```

movq %rax, RAX(%rsp)

```

```

DISABLE_INTERRUPTS(CLBR_ANY)

```

```

TRACE_IRQS_OFF

```

```

movq PER_CPU_VAR(current_task), %r11
testl$_TIF_ALLWORK_MASK, TASK_TI_flags(%r11)
jnz 1f

```

```

LOCKDEP_SYS_EXIT

```

```

TRACE_IRQS_ON /* user mode is traced as IRQs on */

```

```

movq RIP(%rsp), %rcx
movq EFLAGS(%rsp), %r11
RESTORE_C_REGS_EXCEPT_RCX_R11
movq RSP(%rsp), %rsp

```

```

/* Finally USERGS_SYSRET64 macro will be invoked
* This macro expands to the call of the swapgs instruction
* this swapgs instruction will exchanges again the user GS
and kernel GS
* ultimately kernel mode stack is swapped with the user mode
stack.
* finally sysretq will be executed and kernel exit from sytem
call handling
* and user mode program resume exeuction.
*/

```

```

USERGS_SYSRET64

```

```

#define USERGS_SYSRET64 \
    swapgs; \
    sysretq;

```

To sum up, we have deep dived into the Linux system call execution model, right from user space application and syscall instruction from the C library to handling syscall from the kernel perspective. Here are the important points.

- User space application or glibc fills the general-purpose register with system call number as well as arguments for system call and generates syscall instruction.
- Mode switches from the user mode to kernel mode and starts execution of the system call entry code *entry_SYSCALL_64*.
- *entry_SYSCALL_64* switches to the kernel stack and saves general-purpose registers, user mode stack, user mode code segment, flags, etc, onto the kernel stack.
- *entry_SYSCALL_64* checks the system call number in the rax register, searches system call handler in the *sys_call_table* and calls it.
- After a system call handler finishes its work, it restores general-purpose registers, user mode stack, flags, return address, and exits from the *entry_SYSCALL_64* with the *sysretq* instruction. This resumes the execution of the user mode program. **END** 

References

- Understanding the LINUX KERNEL, 3rd edition, by Daniel P. Bovet and Marco Cesati
- Glibc source v2.21
- Linux kernel source v5.0

By: Shwetabh Srivastava

The author is a senior technical architect in the system software group of HCL Technologies Ltd. He has been working on the design and development of Linux based operating systems for a few years.